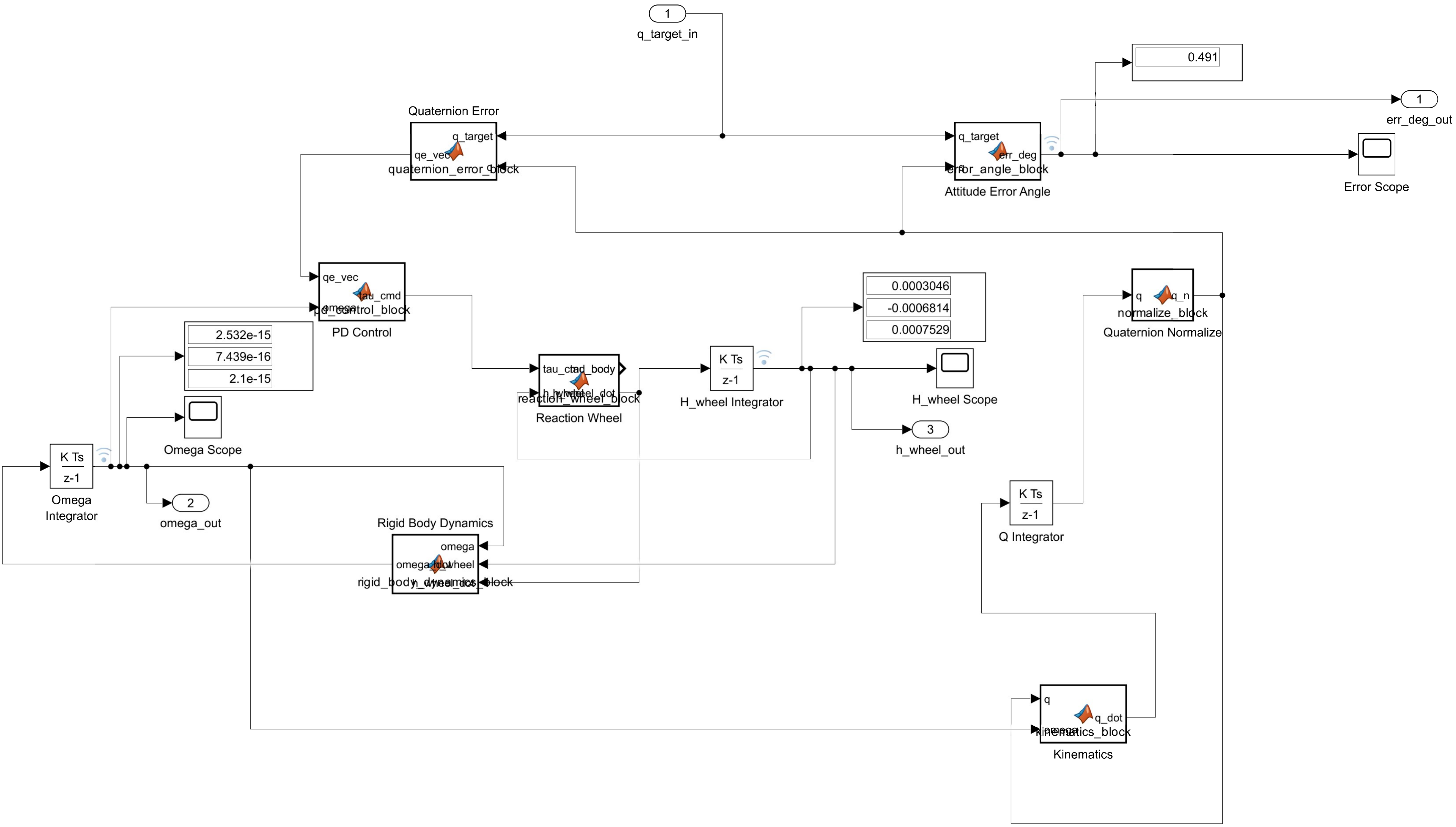

1
q_target_in
Quaternion Error
q_target
qe_vec
q
quaternion_error_block
q_target
err_deg
q
error_angle_block
Attitude Error Angle
0.491
1
err_deg_out
Error Scope
qe_vec
tau_cmd
omega
pd_control_block
PD Control
2.532e-15
7.439e-16
2.1e-15
Omega Scope
K Ts
z-1
Omega Integrator
2
omega_out
tau_cmd
tau_body
h_wheel_dot
reaction_wheel_block
Reaction Wheel
K Ts
z-1
H_wheel Integrator
0.0003046
-0.0006814
0.0007529
H_wheel Scope
3
h_wheel_out
q
q_n
normalize_block
Quaternion Normalize
K Ts
z-1
Q Integrator
Rigid Body Dynamics
omega
omega_dot
h_wheel
tau_body
rigid_body_dynamics_block
q
q_dot
omega
kinematics_block
Kinematics

```
function err_deg = error_angle_block(q_target, q)
    q0 = q(1); qv = q(2:4);
    r0 = q_target(1); rv = -q_target(2:4);
    qe0 = r0*q0 - dot(rv,qv);
    if qe0 < 0, qe0 = -qe0; end
    qe0 = min(max(qe0,-1),1);
    err_deg = 2*acosd(qe0);
end
```

```
function q_dot = kinematics_block(q, omega)
    wx = omega(1); wy = omega(2); wz = omega(3);
    Omega = [ 0 -wx -wy -wz; wx 0 wz -wy; wy -wz 0 wx; wz wy -wx 0 ];
    q_dot = 0.5 * Omega * q;
end
```

```
function tau_cmd = pd_control_block(qe_vec, omega)
    Kp = 3e-3;   % SAME gains as main_pointing.m -- keep in sync if you retune
    Kd = 1.2e-2;
    tau_cmd = -Kp*qe_vec - Kd*omega;
end
```

```
function qe_vec = quaternion_error_block(q_target, q)
    q0 = q(1); qv = q(2:4);
    r0 = q_target(1); rv = -q_target(2:4);  % conjugate of q_target inline
    qe0 = r0*q0 - dot(rv,qv);
    qe_vec = r0*qv + q0*rv + cross(rv,qv);
    if qe0 < 0
        qe_vec = -qe_vec;   % shortest-path correction, same as pid_attitude_control.m
    end
end
```

```
function q_n = normalize_block(q)
    q_n = q / norm(q);
end
```

```
function [tau_body, h_wheel_dot] = reaction_wheel_block(tau_cmd_body, h_wheel)
    h_max = 0.01; tau_max = 0.0006;   % same values as reaction_wheel_model.m
    tau_wheel_cmd = -tau_cmd_body;     % same sign fix as the debugged .m version
    tau_actual = zeros(3,1);
    for i = 1:3
        tau_actual(i) = max(min(tau_wheel_cmd(i), tau_max), -tau_max);
    end
    for i = 1:3
        if (h_wheel(i) >= h_max && tau_actual(i) > 0) || ...
           (h_wheel(i) <= -h_max && tau_actual(i) < 0)
            tau_actual(i) = 0;
        end
    end
    h_wheel_dot = tau_actual;
    tau_body = -tau_actual;
end
```

```
function omega_dot = rigid_body_dynamics_block(omega, h_wheel, h_wheel_dot)
    I = diag([0.04187, 0.04187, 0.006667]);   % same 3U CubeSat inertia as inertia_properties.m
    H_total = I*omega + h_wheel;
    gyroscopic_term = cross(omega, H_total);
    omega_dot = I \ (-gyroscopic_term - h_wheel_dot);   % tau_ext = 0 (pointing loop only, no disturbance)
end
```